

Real-Time Sentiment Analysis

using Apache Spark and Kafka

SECP3133 · High Performance Data Processing

Project 2 · Group Presentation (Group DataDiggers)

Google Play Reviews — Touch 'n Go · Grab · foodpanda



**NUR FIRZANA BINTI
BADRUS HISHAM**
A23CS0156

Data Engineer



**NURAI SYAH BINTI MOHD
ZIKRE**
A23CS0160

Model Engineer



**LUBNA AL HAANI BINTI
RADZUAN**
A23CS00107

Pipeline Engineer



**NURUL IKA SYAFINY BINTI
AZHAR**
A23CS0164

Visualization & Performance
Engineer

Agenda

A

B

C

D

01

Introduction & Background

Problem, objectives, scope

A

05

Analysis, Results & Dashboard

Findings, Kibana visualizations

D

02

Data Acquisition & Preprocessing

Google Play scraping, NLP pipeline

A

06

Optimization & Comparison

Batch vs streaming performance

D

03

Sentiment Model Development

Naive Bayes vs LSTM, evaluation

B

07

Conclusion & Future Work

Takeaways and next steps

A

04

Apache System Architecture

Kafka → Spark → Elasticsearch pipeline

C

01 Introduction & Background

Why this matters



Malaysian digital economy projected at USD 35B by 2025 — app reviews are a real-time pulse of user satisfaction.

The challenge



Traditional batch analytics introduce hours of delay. Organizations need sentiment insight in seconds, not days.

Our solution



End-to-end real-time pipeline: collect → preprocess → classify → stream → visualize, all with open-source tools.

Project Objectives

1

Collect Malaysian-relevant text data from Google Play Store

2

Train & evaluate 2 sentiment models (Naive Bayes + LSTM)

3

Build real-time pipeline with Apache Kafka & Spark

4

Store results in Elasticsearch, visualize in Kibana

5

Compare batch vs streaming performance

02 Data Source — Why Google Play?

Free access, no API key



google-play-scraper library retrieves reviews with country & language filters — no paid subscription needed.

Built-in sentiment labels



Star ratings (1–5) provide free ground-truth labels. No manual annotation required.

Malaysian relevance



Touch 'n Go, Grab, foodpanda — top Malaysian apps with millions of local users.

Large review volume



15,000 reviews collected (5,000 per app) — sufficient for deep learning model training.

15,000

reviews collected

3

Malaysian apps

English (my)

language filter

Star rating

auto-label

02 Text Preprocessing Pipeline

1

Lowercase

ALL → all

2

Noise Removal

URLs, emojis, HTML

3

Tokenize

NLTK word_tokenize

4

Stop Words

Keep: not, no, never

5

Lemmatize

spaCy en_core_web_sm

6

Export CSV

cleaned_data.csv → B

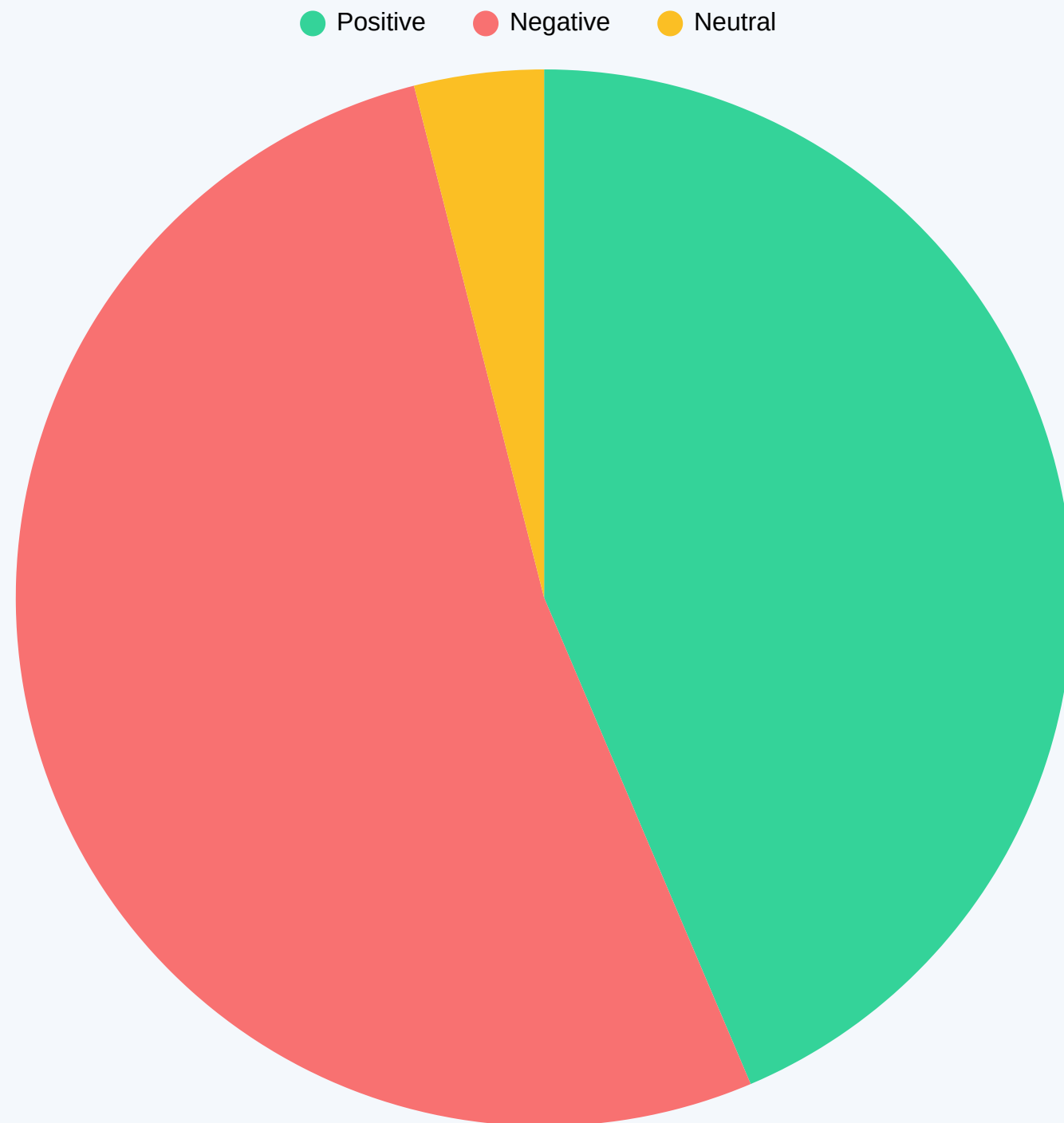
Before → After Example

Raw: "Really BAD app!! Keeps crashing 🤔 http://t.co/abc @grabmy"

Clean: "really bad app keep crash" → Label: negative

02 Final Dataset Overview

Presenter: Firzana



12,567

Total records after cleaning

5,480

Positive (4–5 star reviews)

6,589

Negative (1–2 star reviews)

498

Neutral (3 star reviews)

03 Sentiment Model Development

Model 1 — Naive Bayes

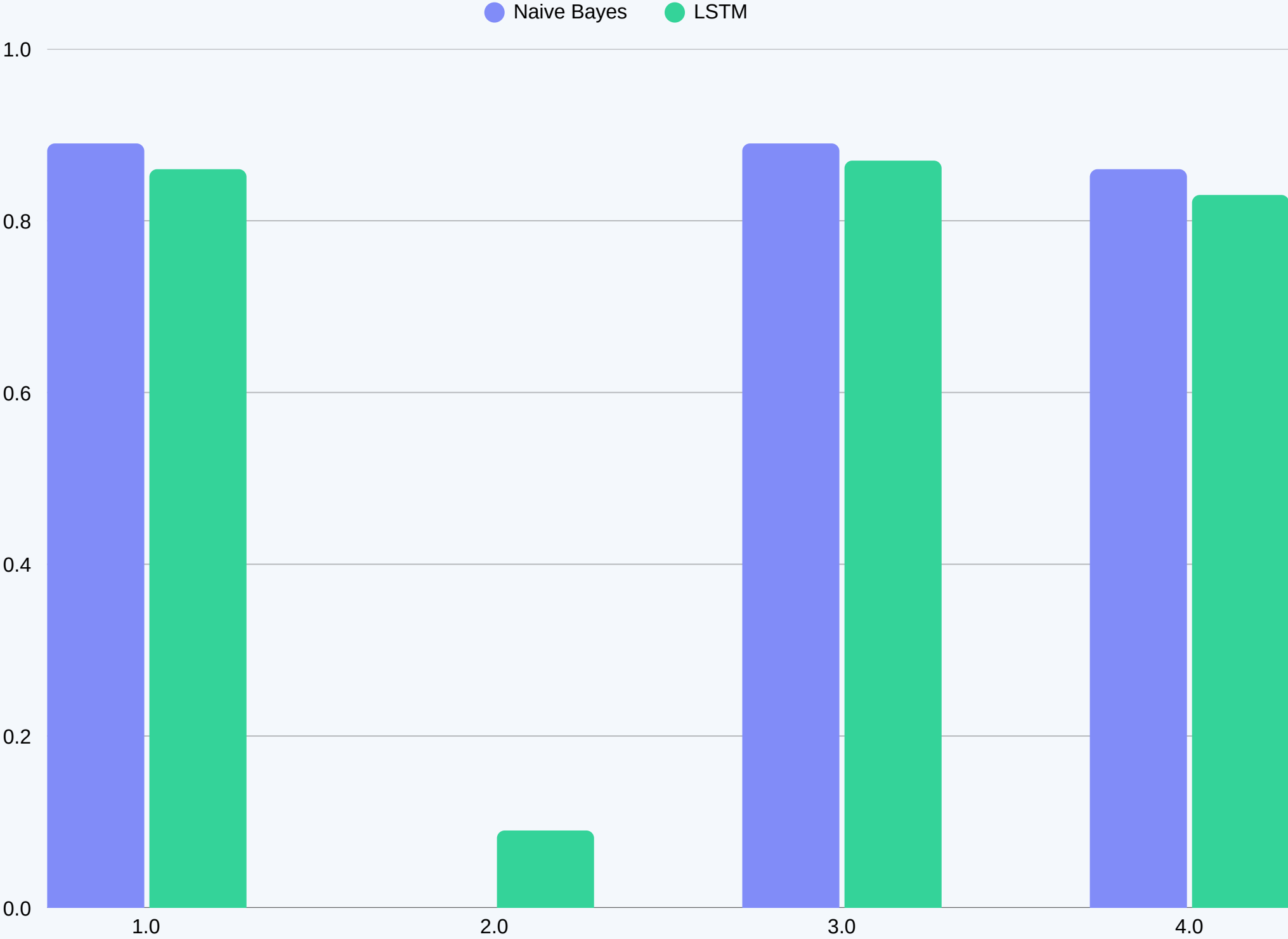
- Library: scikit-learn MultinomialNB
- Features: TF-IDF (20,000 features, unigrams + bigrams)
- Smoothing: alpha = 0.1
- Training time: < 2 seconds
- Dataset split: 70% train / 20% test / 10% val

Model 2 — LSTM

- Library: TensorFlow & Keras
- Architecture: Embedding(64) → LSTM(128) → Dropout → LSTM(64) → Dense(3)
- Optimizer: Adam | Loss: Categorical cross-entropy
- Epochs: 10 | Batch size: 64
- Training time: ~15–25 minutes (CPU)

03 Model Evaluation Results

Presenter: Nuraisyah



Metric	Naive Bayes	LSTM
Overall Accuracy	0.88 ✓	0.84
Negative F1	0.89 ✓	0.86
Neutral F1	0.00 ✗	0.09 ✓
Positive F1	0.89 ✓	0.87
Macro avg F1	0.60	0.61 ✓
Weighted avg F1	0.86 ✓	0.83



Selected for Pipeline: Naive Bayes

Higher overall accuracy (0.88) + microsecond inference = ideal for real-time Spark streaming.

03 Key Insight — Neutral Class Problem



102

neutral test records

Only 4.1% of the test set

Naive Bayes

Precision



0.00

Recall

0.00

F1-score

0.00

Zero neutral detected — entire class suppressed

LSTM

Precision



0.12

Recall

0.07

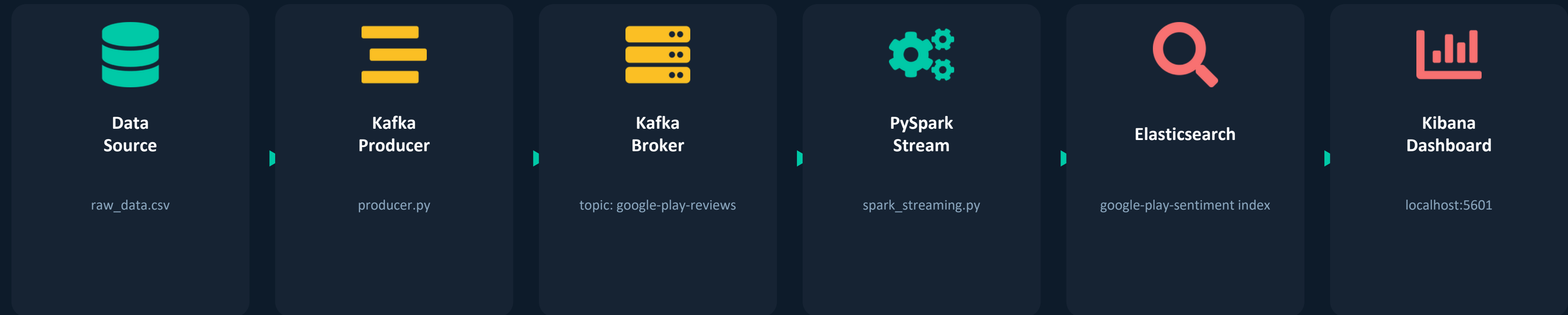
F1-score

0.09

Detected 7 of 102 neutral reviews — marginal improvement

04 Apache System Architecture

Presenter: Haani



All services containerized with Docker Compose – Kafka, Spark, Elasticsearch, and Kibana start with one command: `docker-compose up -d`

Kafka Topic

- Name: google-play-reviews
- Partitions: 3
- Replication: 1

Spark Job

- Trigger: every 5 seconds
- Checkpoint: ./checkpoint
- Output: foreachBatch → ES

Model UDF

- load NB model + TF-IDF
- predict_sentiment(text)
- Applied per micro-batch

Elasticsearch

- Index: google-play-sentiment
- Port: 9200
- Security: disabled (dev)

04 Inside the Spark Streaming Job

```
# 1. Read from Kafka

raw = spark.readStream.format('kafka').option('subscribe','reviews-stream').load()

# 2. Parse JSON → text field

parsed = raw.select(from_json(col('value').cast('string'),
schema).alias('data')).select('data.*')

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"@w+", "", text)
    text = re.sub(r"^[w\s]", "", text)
    text = re.sub(r"\d+", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

# 3. Apply model UDF (Naive Bayes + TF-IDF)

classified = parsed.withColumn('sentiment', sentiment_udf(col('text')))

                    .withColumn('processed_at', current_timestamp())

# 4. Write to Elasticsearch every 5 seconds

query = classified.writeStream.foreachBatch(write_to_es)

                    .trigger(processingTime='5 seconds').start()
```

1

Producer sends data

kafka_producer.py reads raw_data.csv row by row, sends JSON messages to Kafka at 100 rec/sec.

2

Spark consumes stream

PySpark reads the Kafka topic as a continuous stream, parsing JSON payloads into a DataFrame. Data was cleaned and preprocessed, including text normalization.

3

Model classifies

A UDF loads the Naive Bayes model + TF-IDF vectorizer and applies predict_sentiment() to each row in parallel.

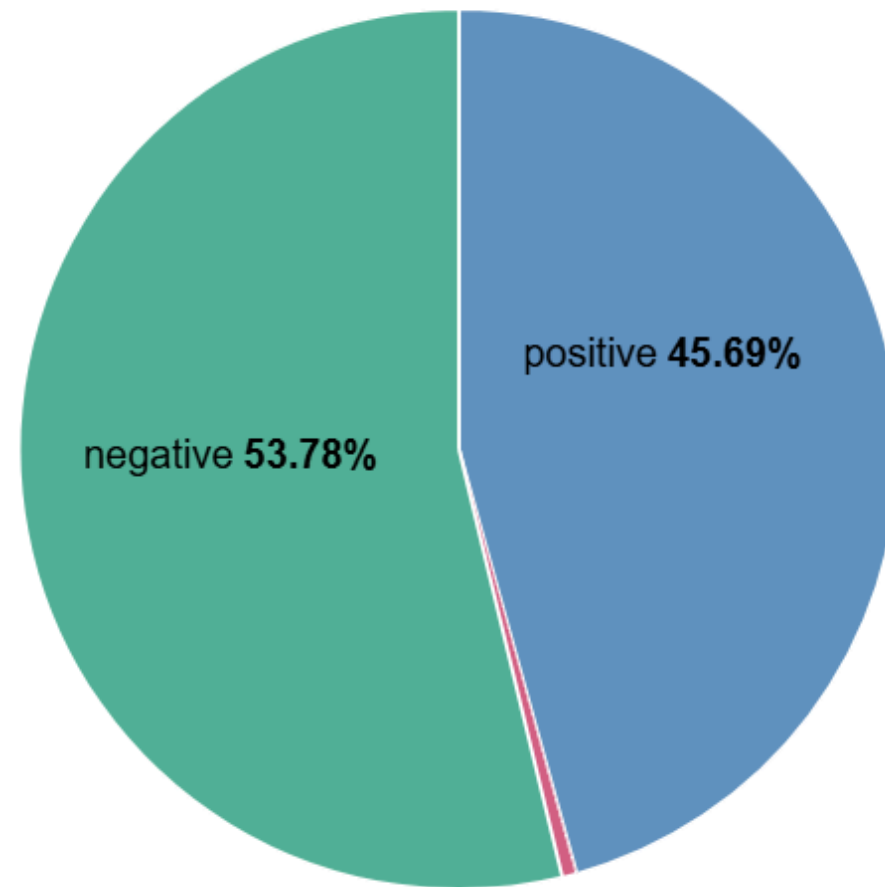
4

Results stored in ES

Every 5-second micro-batch is written to the sentiment_results Elasticsearch index for querying.

05 Analysis & Results — Key Findings

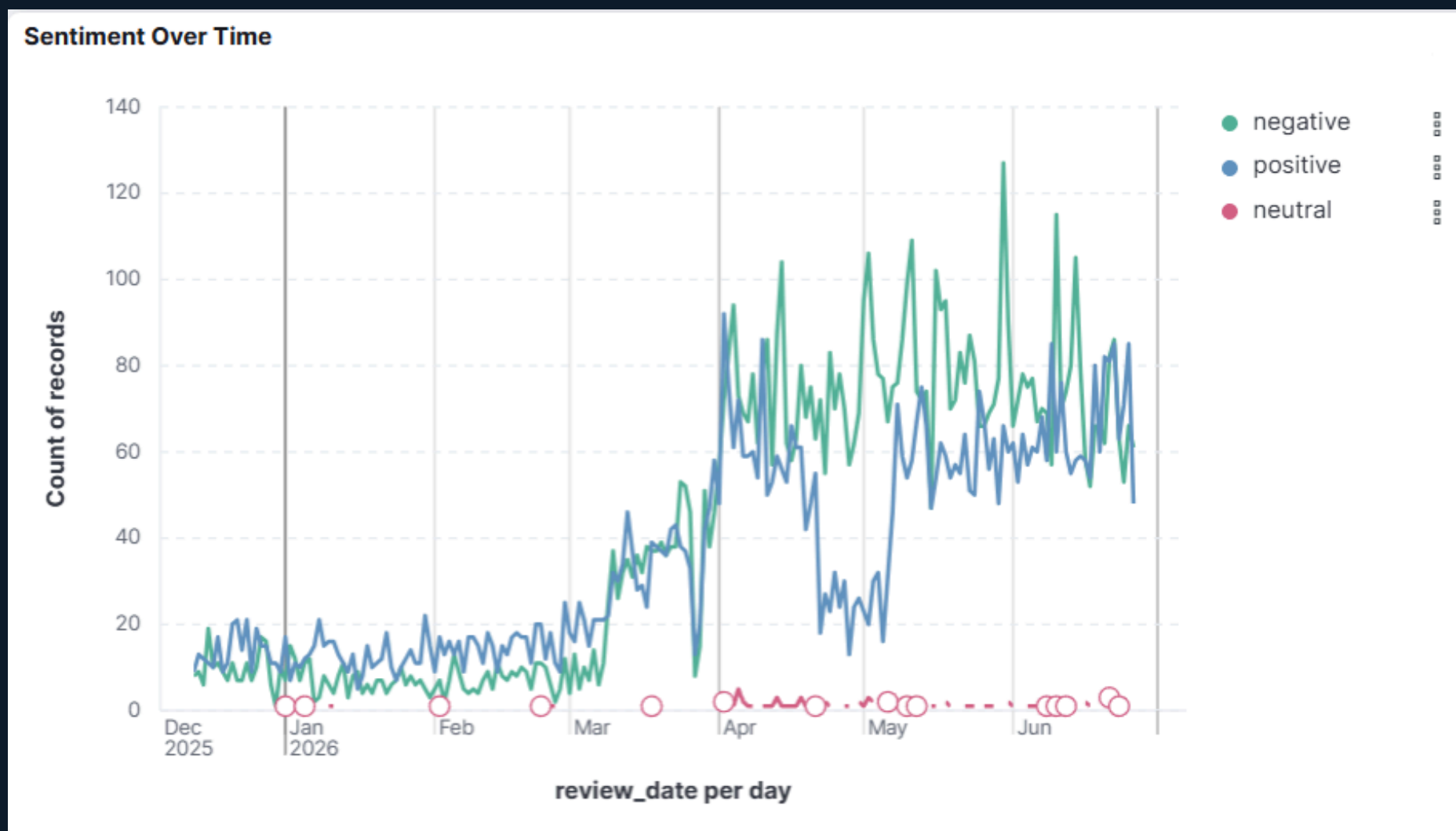
Sentiment Destribution



Sentiment Distribution (Pie Chart)

- 45.69% positive, 53.78% negative, 0.53% neutral
- Negligible neutral share reflects the typical app-review pattern: users write reviews to praise or complain, rarely to stay neutral
- Overall skew toward negative still signals more dissatisfaction than satisfaction across the streamed dataset

05 Analysis & Results — Key Findings



Sentiment Over Time (Line Chart)

- Dec 2025–Mar 2026: low, stable volume (~5–20 reviews/day), positive $\approx$ negative
- Late Mar/early Apr 2026: sudden volume spike, negative overtakes positive
- Negative sustains 40–130 reviews/day through June 2026 (peak $\sim$ 127)
- Timing + volume jump suggests a specific triggering event like app update, pricing change, service disruption.

05 Analysis & Results — Key Findings

Sentiment Breakdown By App			
Top 5 values of sentiment	Top 5 values of app	Count of records	Average of rating
negative	foodpanda	3,611	1.354
negative	grab	2,510	1.778
negative	touch_n_go	1,946	2.341
positive	touch_n_go	3,032	4.838
positive	grab	2,441	4.827
positive	foodpanda	1,380	4.695
neutral	grab	49	2.755
neutral	touch_n_go	22	3
neutral	foodpanda	9	3

Sentiment Breakdown by App (Table)

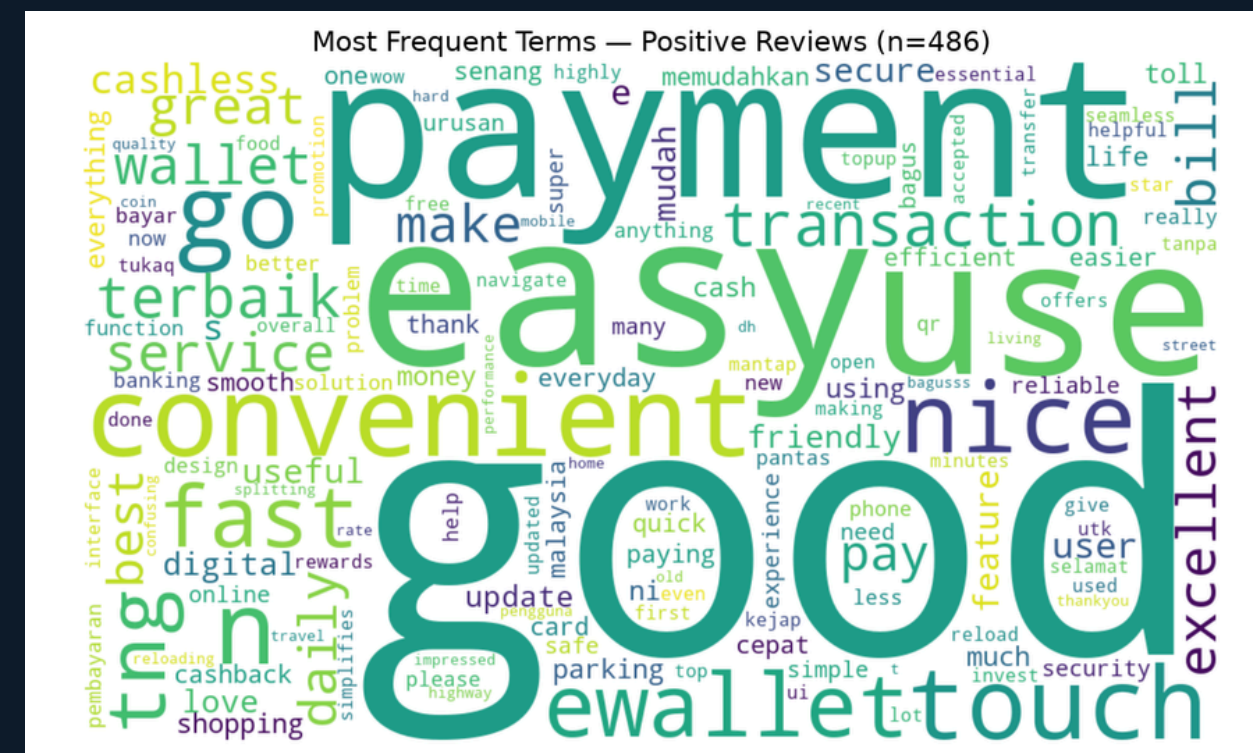
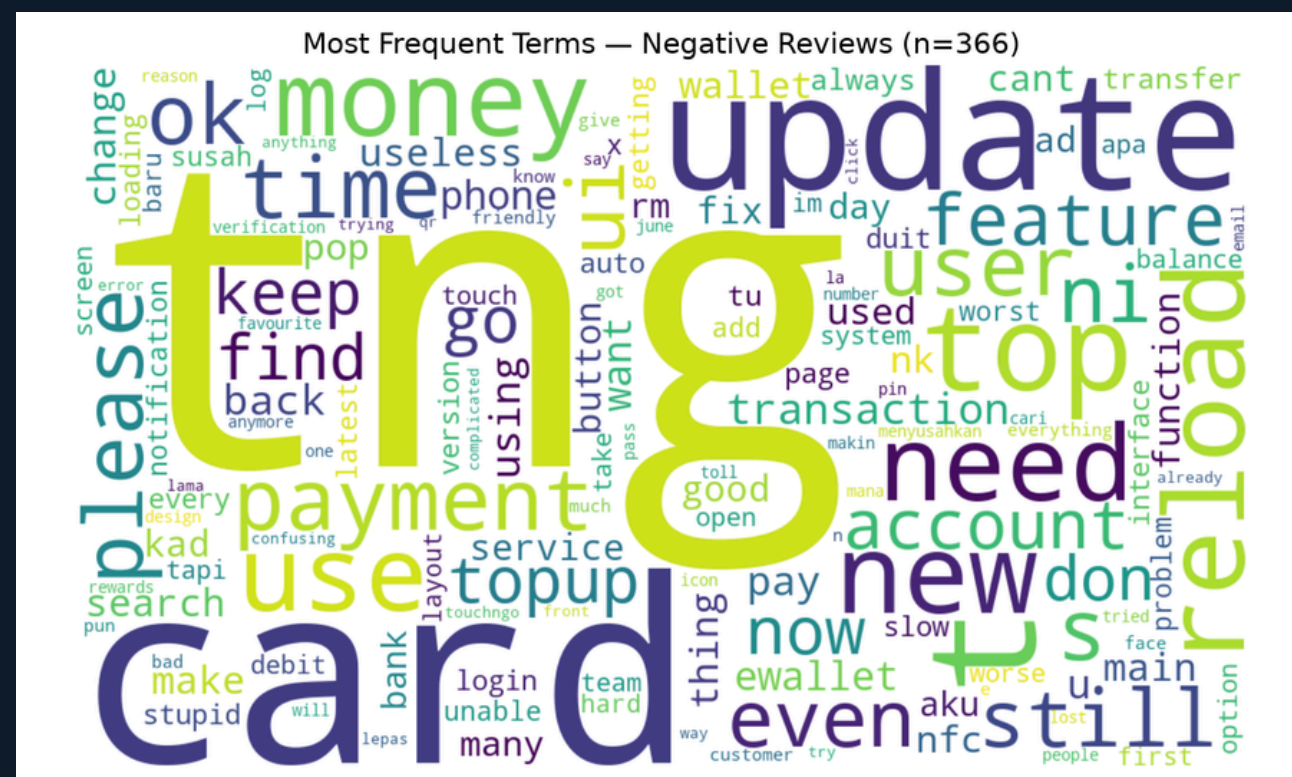
- Foodpanda: most negatively skewed (3,611 negative vs. 1,380 positive, avg. rating 1.30)
- Touch 'n Go: net positive (3,032 positive vs. 1,946 negative)
- Grab: near-balanced (2,441 positive vs. 2,510 negative)
- Foodpanda's skew is what drags the overall pie chart negative
- Star ratings cross-validate sentiment labels (positive avg. 4.63–4.84★, negative avg. 1.3–2.34★)

05 Analysis & Results — Key Findings



Negative Reviews

- update, card, reload, topup, payment, account + useless, stupid, confusing, unable → points to a specific update that broke/complicated the reload/top-up flow
- Malay terms (susah, baru, makin, lama) show the frustration crosses both language groups

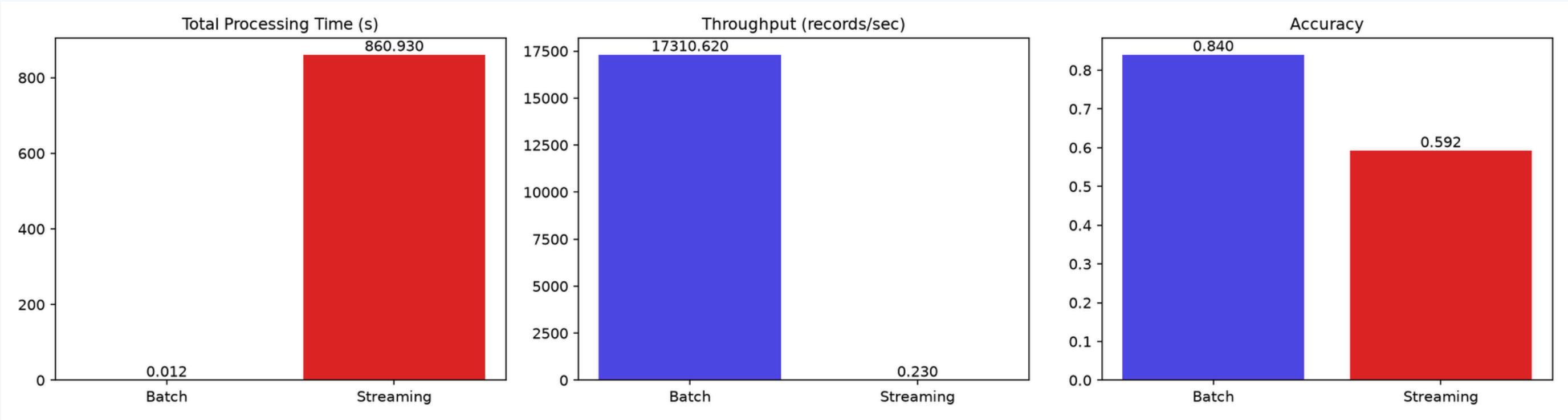


Positive Reviews

- easy, convenient, fast, reliable, secure, cashback + Malay (mudah, senang, bagus) → generic, stable satisfaction with the core e-wallet experience, not tied to one feature
- Conclusion: negative surge is specific & actionable (reload/top-up UX), not a general decline

06 Optimization — Batch vs Streaming Comparison

Metric	Batch Mode	Streaming Mode
Total records	200	200
Total time	0.012s	860.93s
Throughput (rec/s)	17,310.62 rec/s	0.23 rec/s
Accuracy	0.840	0.592*
CPU usage	0.0%†	6.64%†
Memory	0.32MB†	1.05MB†



06 Optimization — Batch vs Streaming Comparison

Optimization Opportunities

- Bulk ES indexing — replace per-row `es.index()` with bulk API to cut network round-trips
- Exact-match evaluation — added `review_id` through Kafka → Elasticsearch doc ID, removes row-order assumption, makes pipeline safely re-runnable
- Explicit index mapping — apply `elastic_mapping.json` before first write to avoid auto-detected wrong field types (e.g. `review_date` as text, not date)
- Configurable producer delay — make the 1.5s delay a CLI argument for both demo pacing and max-throughput benchmarking

07 Conclusion & Future Work

What We Built



Collected 12,567 Malaysian Google Play reviews across 3 apps



Trained Naive Bayes (acc 0.88) and LSTM (acc 0.84) — NB selected for pipeline



Built end-to-end Kafka → Spark → Elasticsearch streaming pipeline



Created 4-panel Kibana dashboard with real-time sentiment visualization



Compared batch vs streaming — same accuracy

Future Work

1

Add Bahasa Malaysia & Manglish support using multilingual transformer models

2

Address neutral class imbalance (4.1%) using SMOTE and DistilBERT fine-tuning

3

Integrate live X (Twitter) API for real-world social media streaming

4

Scale to AWS EMR or Databricks for production-grade throughput (> 50K rec/s)

Thank You